

Investigación Conceptual.

Estudiante: Edward Camaño – Cédula: 8-1010-515 – Grupo: 1SF241

Conceptos Elementos de la arquitectura

1. Arquitectura Cliente-Servidor: Modelo donde el cliente solicita servicios y el servidor los provee vía red (ej. HTTP, FTP). Separa responsabilidades.
2. IP Address: Etiqueta numérica única (IPv4 o IPv6) que identifica cada dispositivo en una red para permitir la comunicación.
3. DNS (Domain Name System): "Agenda telefónica" de Internet. Traduce nombres de dominio (ej. google.com) a direcciones IP.
4. Proxy/Reverse Proxy: Un proxy inverso se ubica frente a los servidores. Provee equilibrio de carga, caché, seguridad (oculta IPs) y monitoreo.
5. Latencia: Retraso (en ms) entre una acción del usuario y la respuesta del sistema. La distancia y los componentes de red son causas principales.
6. HTTP/HTTPS: HTTP es el protocolo de transferencia de hipertexto. HTTPS es su versión segura, que cifra los datos con certificados SSL/TLS.
7. APIs (Interfaz de Prog. de Aplicaciones): Conjunto de definiciones que permite que dos componentes de software se comuniquen (ej. SOAP, RPC, WebSocket, REST).
8. REST API: Estilo arquitectónico que usa solicitudes HTTP (GET, POST, PUT, DELETE) para operaciones CRUD sobre recursos, generalmente usando JSON.
9. GraphQL: Lenguaje de consulta que permite al cliente solicitar exactamente los datos que necesita en una sola consulta, evitando múltiples viajes.
10. Base de Datos: Almacén organizado de información (ej. relacional, jerárquica, orientada a objetos) gestionado por un DBMS.
11. SQL vs NoSQL: SQL es relacional, estructurado y con esquema fijo (ideal para integridad). NoSQL es no relacional, flexible y escalable (ideal para grandes volúmenes y datos no estructurados).
12. Escalado Vertical ("Scale Up"): Aumentar recursos (CPU, RAM) de un mismo servidor. Simple pero con límite y costoso.
13. Escalado Horizontal ("Scale Out"): Añadir más servidores (nodos) al sistema. Más complejo pero muy potente y sin límite teórico.
14. Equilibrio de Carga (Load Balancer): Distribuye el tráfico entrante entre múltiples servidores para evitar sobrecargas y maximizar rendimiento.

15. Indexación: Estructura que organiza datos (ej. palabras clave) para acelerar drásticamente las búsquedas y filtros (a costa de espacio y escritura más lenta).

16. Replicación: Mantener múltiples copias de los mismos datos en diferentes ubicaciones. Mejora disponibilidad, resiliencia y reduce latencia (sincrónica o asincrónica).

17. Fragmentación (Sharding): Datos dispersos en diferentes sistemas. Un desafío que genera islas de datos; se combate con integración, gobierno y arquitecturas unificadas.

18. Particionamiento Vertical: Dividir una tabla por columnas (no por filas). Aísla columnas según frecuencia de acceso para optimizar I/O y caché.

19. Almacenamiento en Caché (Caching): Guardar copias temporales de datos (ej. en memoria) cerca del usuario para acelerar accesos futuros. Reduce latencia y carga del servidor.

20. Desnormalización: Añadir redundancia deliberada a un esquema normalizado para acelerar lecturas y simplificar consultas (a cambio de escrituras más complejas).

21. Teorema CAP: En un sistema distribuido, solo se pueden garantizar 2 de 3: Consistencia, Availability (Disponibilidad), Partition Tolerance (Tolerancia a Particiones). Las bases de datos se clasifican en CP, AP o CA.

22. Almacenamiento de BLOB (Binary Large Object): Almacenamiento en la nube para datos no estructurados (imágenes, backups). Usa un modelo plano ("data lake") sin jerarquía.

23. CDN (Content Delivery Network): Red de servidores geográficamente dispersos que almacenan copias (caché) de contenido estático y dinámico cerca del usuario final, reduciendo latencia.

24. WebSocket: Protocolo que mantiene una conexión dúplex (bidireccional) y persistente entre cliente y servidor, ideal para apps en tiempo real (ej. chat, trading).

25. Webhooks: Comunicación ligera basada en eventos. El servidor empuja datos al cliente (vía HTTP POST) cuando ocurre un evento específico ("API inversa").

26. Microservicios: Arquitectura donde una aplicación se compone de múltiples servicios pequeños, independientes, con su propia base de datos y tecnología.

27. Cola de Mensajes: Middleware que almacena mensajes de forma asíncrona hasta que el consumidor los procesa. Garantiza resiliencia y desacoplamiento.

28. Limitación de Velocidad (Rate Limiting): Técnica que regula la frecuencia de solicitudes entrantes para prevenir abusos, DoS y garantizar equidad.

29. Puertas de Enlace de APIs (API Gateway): Punto de entrada único que gestiona, enruta, protege y monitorea las solicitudes a múltiples servicios de backend (esencial en microservicios).

30. Idempotencia: Propiedad de una operación que produce el mismo resultado si se ejecuta 1 o N veces. Permite reintentos seguros sin efectos secundarios (clave en pagos y colas).